SDR Hunter — Hardware Bring-Up Guide

This guide takes you from a freshly-plugged-in SDR to a validated, running SDR Hunter install. It is meant to be run **on the machine the SDR is physically attached to** — not in a headless/mock environment. Every step is ordered so that if something fails, you stop at the lowest layer that's broken instead of chasing a symptom three layers up.

> Legend: `$` = run as your normal user, `#` = needs *sudo*/root.
> Supported hardware: **BladeRF, RTL-SDR / NooElec, HackRF, LimeSDR, PlutoSDR, USRP (UHD), SDRplay, Airspy** — all via SoapySDR.

Layer 0 — Prerequisites & driver install

1. **Install SoapySDR + per-device drivers + udev rules** using the bundled installer (Debian/Ubuntu, Fedora, Arch, macOS all handled):

```
bash
  $ cd /path/to/sdr_hunter
  $ ./install_drivers.sh
```

This installs `soapysdr-tools`, the Python bindings (`python3-soapysdr`), and the driver module for each supported radio. **SoapySDR Python bindings come from the system package, NOT pip** — do not `pip install` them.

1. **Install the app's Python deps:**

```
bash
  $ pip install -r requirements.txt
  # optional decoders/extras (weather-sat, drone-id, etc.):
  $ pip install -r requirements_optional.txt
```

1. **Re-plug the device after install** so the new udev rules apply, OR reload them without unplugging:

```
bash
  # udevadm control --reload-rules && udevadm trigger
```

Layer 1 — Is the OS seeing the USB device?

Before any SDR software, confirm the kernel enumerated the hardware.

```
$ lsusb
```

Look for your radio in the list, e.g.:

Device	Typical `lsusb` string
RTL-SDR	`Realtek ... RTL2838 DVB-T`
HackRF	`Great Scott Gadgets HackRF One`
BladeRF	`Nuand ... bladeRF`
LimeSDR	`Lime Microsystems LimeSDR-USB`
Airspy	`Airspy`
PlutoSDR	`Analog Devices Inc. PlutoSDR (ADALM-PLUTO)`

If it does not appear: it's a cable/power/USB-port problem, not a software one. Try a different (data-capable) cable, a rear/powered USB port, and avoid unpowered hubs. BladeRF/LimeSDR/USRP draw significant current — use USB 3.0.

> **RTL-SDR only:** blacklist the kernel DVB-T driver or it will grab the device:
> ```bash
>
> # echo 'blacklist dvb_usb_rtl28xxu' > /etc/modprobe.d/blacklist-rtl.conf
>
> # modprobe -r dvb_usb_rtl28xxu # (or replug)
>
> ```

Layer 2 — Does SoapySDR see it?

This is the layer SDR Hunter actually talks to. Enumerate with the SoapySDR tool directly:

```
$ SoapySDRUtil --find
```

You should see a probe result with a `driver=` key (e.g. `driver=bladerf`, `driver=rtlsdr`). Full capability probe (sample-rate ranges, gains, bandwidths):

```
$ SoapySDRUtil --probe="driver=<yourdriver>"
```

If `lsusb` shows the device but `SoapySDRUtil --find` does not: the matching SoapySDR **module** isn't installed or isn't on the module path. Re-run `./install_drivers.sh`, then check `SoapySDRUtil --info` lists your driver under "Available factories".

Permissions check: if `--find` only works under `sudo`, the udev rule didn't apply. Fix the rules (Layer 0 step 3) and replug — **do not** routinely run the app as root.

Layer 3 — Does SDR Hunter see it?

Now use the app's own enumeration, which goes through its `DeviceManager`:

```
$ python3 main.py --list-devices
```

Expected: `SoapySDR available: True` and your device listed with driver/label/serial. If it prints the **synthetic/mock** device instead ("No hardware/SoapySDR detected; using synthetic device"), SoapySDR isn't importable from this Python — you're likely in a virtualenv that can't see the system `SoapySDR` bindings. Either use the system Python, or create the venv with `--system-site-packages`.

For a deeper, layer-by-layer automated check, run the bundled diagnostic:

```
$ python3 tools/bringup_check.py
$ python3 tools/bringup_check.py --probe   # also does a short capture per device
```

Layer 4 — First real capture (smoke test)

Launch the desktop GUI against real hardware (drop the mock env vars — they force the synthetic device):

```
$ ./launch.sh
# equivalently: python3 main.py --gui
```

In the **Device panel**:

1. Select your real device (not "Mock").
2. Set a **center frequency** on a known-active band — FM broadcast **88–108 MHz** is the easiest sanity check.
3. Set **Sample rate** and **Bandwidth** within the device's range (start conservative: 2.0–2.4 MS/s, 2.0 MHz bandwidth). The Bandwidth control clamps to your device's profile automatically.
4. Set **gain** to ~30–40 dB (or enable AGC) and start the stream.

Success looks like: a live spectrum with a noise floor and, on the FM band, clear station peaks; the waterfall scrolls. Tune to a strong FM station and the FM decoder should produce audio.

If the spectrum is flat/dead: wrong antenna for the band, gain too low, or the device is streaming but not tuned where you think. **If it's a wall of noise:** gain too high (clipping) or you're near strong interference — reduce gain.

Layer 5 — Dual-RX, web dashboard & tuning

1. **Web dashboard:** `python3 main.py --web` (or `--both` for GUI + web), then open `http://localhost:8000`. Confirm the WebSocket status indicators go green (`/ws/spectrum`, `/ws/events`, `/ws/waterfall`).
2. **Remote tune / RX routing:** in the web tune form pick **RX1** (scanner) vs **RX2** (focus) and a frequency, then Focus. RX1 parks the scanner on that frequency (halting the sweep); `release` resumes the sweep. RX2 tunes the focus receiver independently. Verify each RX behaves as expected on real hardware. (Bandwidth is selectable in the desktop Device panel; the web tune form does not yet expose a bandwidth field.)
3. **Two-device setup:** for genuine simultaneous dual-RX, attach two radios (or one 2-channel device like LimeSDR/USRP). Confirm both appear in `--list-devices` with distinct serials.

Layer 6 — Decoders & ATAK (as needed)

- **NOAA APT weather-sat:** needs a real pass on 137 MHz with an appropriate antenna (V-dipole/QFH). Decoder runs at 20800 Hz, sync-aligned, false-color composite.
- **Drone ID:** tune the drone-ID bands and confirm detections populate the drone map.
- **ATAK bridge:** open the ATAK config dialog, set multicast/unicast + callsign
- stale time, and use **Test ping** to confirm your ATAK/WinTAK endpoint receives CoT before relying on live detections.

Quick troubleshooting matrix

Symptom	Most likely layer	First thing to check
Device missing from `lsusb`	1 (USB)	Cable (data, not charge-only), powered USB 3 port
In `lsusb`, not in `SoapySDRUtil --find`	2 (Soapy module)	Re-run `install_drivers.sh`; check factories
Works only with `sudo`	0/2 (udev)	Reload udev rules + replug
App shows mock despite hardware present	3 (Python env)	venv can't see system SoapySDR bindings
Flat/dead spectrum	4 (RF)	Antenna for band, gain, actual tuned freq
Wall of noise	4 (RF)	Gain too high / nearby interference
Web WS indicators red	5 (server)	Server running? Port 8000 free? firewall?

What can be validated here vs. only on your machine

This environment is **headless with a mock device**, so everything below the RF layer (imports, enumeration fallback, API routing, decoder unit paths) is already validated. **Layers 1, 2, and 4–6 require the physical radio and RF signals and can only be confirmed on your hardware.** Report back what `--list-devices` and `tools/bringup_check.py` print if anything doesn't line up with the expected output above.